DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Om nom nom!

CMSC 197 GDD Machine Problem 2

Prepared by: Rene Andre B. Jocsing

Published: March 02, 2026 **Deadline:** April 01, 2026

Your second machine problem is to create Yet Another Feeding Frenzy Clone (YAFFC) from scratch in Godot. You may use free-to-use art and sound assets, but the implementation and code must come from you.

Learning Objectives

By accomplishing this machine problem, you should be able to:

- Design and implement a real-time size-comparison system with continuous state evaluation
- Manage a dynamic entity population using object pooling or deferred instantiation
- Apply hierarchical FSMs to manage both individual entity behavior and global game state
- Implement scaling and visual feedback tied to numerical game state
- Deepen your understanding of signals, groups, and scene composition in Godot
- Demonstrate data-driven design by externalizing tunable parameters from logic
- Practice incremental, commit-by-commit development on a non-trivial feature set

---

Why Feeding Frenzy?

Feeding Frenzy looks like a simple fish game. You swim around, you eat things smaller than you, you avoid things bigger than you, and you grow. That's the whole game—or so it seems.

What makes Feeding Frenzy an excellent second machine problem is that it forces you to think about *emergent complexity*. There is no scripted level layout, no handcrafted pipe arrangement. The challenge arises entirely from the interaction of many independent entities operating by the same simple rule: eat what is smaller, flee from what is larger. When you have dozens of fish on screen, each governed by that rule, and the player's size is constantly shifting, the game becomes a dynamic system rather than a static obstacle course.

This is a fundamental shift from Flappy Bird. In Flappy Bird, the world was deterministic—pipes spawn on a timer, the gap is randomized once, and the challenge is fixed at design time. In Feeding Frenzy, the world is *alive*. Your architecture has to reflect that. You can't just add a `fish` variable and call it a day. You need to think about how entities are spawned, how they discover each other, how they are removed from the world without leaving dangling references, and how size—a continuous numeric value—becomes the axis around which every game interaction is decided.

By the time you're done, you'll have built a small but genuine simulation with emergent behavior, real-time continuous state, and a codebase that would survive being handed to another developer without apology.

The Original Mechanics

The Player Fish

The player controls a fish that swims freely in a bounded ocean environment. Movement should feel fluid and responsive—the fish should not snap to directions, but should turn and accelerate with some inertia. This matters more than it sounds. A fish that teleports to the cursor feels wrong. A fish with excessive drag that takes three seconds to stop feels wrong. The right balance is a fish that feels alive: it has weight, it has momentum, and it responds predictably to input. Animation-wise, this manifests as *smear frames*.

At minimum, support mouse-driven movement (the fish swims toward the cursor), eight-directional keyboard movement, or joystick input from a controller. You may support whatever control schema. Screen boundaries should constrain the player—the fish cannot swim off-screen, but it should not hard-stop at the wall either. A soft boundary (the fish slows and turns as it approaches the edge) feels far better than an invisible wall.

The Size System

This is the heart of the game, and it deserves more thought than a single `if` statement.

Every entity in the game—player and NPC alike—has a size value. This value determines three things simultaneously: what the entity can eat, what can eat the entity, and how the entity *looks*. Size should be a continuous float, not a discrete integer level. When the player eats a fish, their size increases by some fraction of the consumed fish's size. The player's sprite (and collision shape) should scale smoothly to reflect this growth.

The comparison rule is straightforward: an entity can eat another if its size exceeds the target's size by a defined threshold. This threshold is a tunable parameter—do not hard-code it. If the threshold is too tight, players will feel cheated by ambiguous collisions. If it is too loose, the game loses tension because everything is edible. Expose this as an exported variable and tune it during playtesting.

On Scaling Collision Shapes

When you scale a sprite, your `CollisionShape2D` does not automatically scale with it if the scale is applied to the sprite node rather than the parent. Be deliberate about your scene hierarchy. Scale the root node of each fish, or update the collision shape's extents programmatically when size changes. Either approach is valid, but pick one and be consistent. Mixing both is a reliable way to introduce subtle, hard-to-debug collision errors.

NPC Fish

The ocean should be populated with fish of varying sizes that swim across the screen. At a minimum, NPC fish should:

- Spawn off-screen and despawn after exiting the opposite boundary
- Move at a speed that loosely correlates with their size (smaller fish are faster)
- Flee from the player if the player is large enough to eat them
- Chase the player if they are large enough to eat the player
- Eat smaller NPC fish they collide with (this gives the world life even when the player isn't involved)

The last point is crucial. A world where NPC fish ignore each other feels like a shooting gallery. A world where they prey on each other feels like an ecosystem.

NPC behavior does not need to be sophisticated. Simple steering—flee if threatened, chase if predatory, wander otherwise—is sufficient. But it must be implemented as a proper FSM per entity, not as a chain of nested `if` statements in `_process()`.

Spawning and Population Management

Fish should spawn continuously throughout the game. A spawn manager (a dedicated node or autoload) is responsible for maintaining the population. Consider these parameters:

- Target population (how many fish should be on screen at any given time)
- Size distribution (spawn weights for small, medium, and large fish)
- Spawn rate (how frequently new fish are introduced)
- Spawn margin (how far off-screen fish spawn to avoid popping into view)

These values should all be exported constants or configuration resources—not magic numbers buried in `_ready()`. The difficulty of the game is directly controlled by these parameters. Early in the session, the ocean should be dominated by small, edible fish. As the player grows, the population should shift. You may implement this as a gradual bias change over time, or as a response to the player's current size.

Object pooling is not required, but instantiating and freeing nodes continuously has a cost. If you notice frame drops during heavy spawning, investigate pooling. At minimum, ensure you are using `call_deferred("queue_free")` or equivalent to avoid freeing nodes mid-physics step.

Scoring and Progression

The player earns points for each fish consumed, scaled by the size of the fish eaten. Eating a fish twice your size (if your twist allows it) should be worth more than eating a fish half your size. Eating a fish of comparable size should feel like a meaningful accomplishment.

Display the current score prominently alongside the player's current size or a visual progress indicator toward the next "tier" of fish they can safely eat. When the player is eaten, the game ends cleanly—no lingering signals, no orphaned nodes, no memory that the player was ever there.

Game States

Your game requires at minimum three states managed by a top-level FSM:

- **Menu/Idle State**: The initial state before gameplay begins
- **Playing State**: Active gameplay with input, physics, and spawning
- **Game Over State**: Triggered when the player is consumed; displays final score and restart option

A fourth state—**Level Complete** or **Win State**—is required if your design has a defined win condition (e.g., reaching a target size). If your game is pure survival with no win condition, three states are sufficient.

FSMs for individual NPC fish (wander, flee, chase) are separate from the global FSM. Do not conflate the two. The global FSM governs the game session. Entity FSMs govern individual behavior. Review your work from the Coin Dash and Jungle Jump activities for reference.

Your Original Mechanics

What Makes a Valid Twist?

Your clone must include a **unique mechanical twist (or several)** that meaningfully changes gameplay. This is NOT cosmetic.

Valid Examples:

- Predator fishes that hunt purely by sound—the player must stay still to avoid detection, turning it into a stealth game
- A venom system where certain fish poison the player on contact, temporarily shrinking them, and maybe the player can shrink other fish, too
- A co-op or competitive split-screen mode with player fishes in the same ocean, the game scaling according to player count

- Depth layers (foreground, midground, background) that the player can shift between to access different prey
- An ecosystem economy: overeating causes smaller fish to stop spawning, starving the player and forcing them to subdue bigger fish

Invalid Examples:

- Different fish sprites or an underwater visual theme only
- Adding background music or ambient sound effects
- Changing the UI layout or color palette

Ask yourself: Does this change how I play the game, or just how it looks or sounds? Will I get struck for copyright infringement if I publish this on Steam or Google Play?

You may deviate from the Technical Requirements section if and only if: it benefits or is crucial to your original mechanics, or is architecturally sound with justification!

Technical Requirements

Architecture

- **Strong typing:** All function signatures and class members must be type-annotated (whether explicitly through Kotlin-style typehints or implicitly through Python-style walrus operators)
- **State machines:** Enum-based FSM for global game state; separate FSM per NPC entity for behavior states
- **Scene composition:** Separate scenes for Player, NPC Fish, SpawnManager, UI, Main, and any HUD elements
- **Signal-driven:** Use signals for decoupled communication (fish eaten, size changed, game over, score updated)
- **Data-driven configuration:** Tunable parameters must be exported variables or `Resource` configurations, not inline literals
- **Separation of concerns:** Movement, AI steering, size logic, and collision handling live in distinct methods and/or nodes

Common Pitfalls

Spaghetti code will cap your grade at 3.00 regardless of gameplay quality. Architecture is heavily weighted in grading. A monolithic `Main.gd` that handles spawning, physics, scoring, and AI simultaneously is not acceptable—even if the game runs perfectly.

Input Handling

Support at least one input method for controlling the player fish. Mouse-based movement (swim toward cursor) is the most natural and is strongly recommended as your primary control scheme. Keyboard movement is a valid alternative or supplement. Touch input or controller input is a valid stretch goal.

Ensure that input is processed only during the Playing state. The fish must not respond to input during the Menu or Game Over states. Only the UI responds here.

Collision Detection

Implement collision detection for all of the following interactions:

- Player eating a smaller NPC fish
- Player being eaten by a larger NPC fish
- NPC fish eating smaller NPC fish
- Player or fish colliding with screen boundaries

Use `Area2D` nodes with collision shapes. Scale collision shapes consistently with entity size. Layer and mask configuration in the Physics settings is your friend here—set up collision layers so that fish-to-fish detection and fish-to-boundary detection are properly isolated.

Entity Behavior (NPC AI)

Each NPC fish must operate under its own simple FSM with at minimum three states: **Wander**, **Flee**, and **Chase**. Transitions are triggered by proximity and size comparisons:

- If a threat (larger fish) enters detection range: transition to **Flee**
- If prey (smaller fish) enters detection range: transition to **Chase**
- Otherwise: **Wander** (move in a general direction with occasional mild steering)

Detection range should be a tunable exported variable. Avoid polling every entity on every frame—use `Area2D` overlap signals to trigger state transitions. This is both more performant and more architecturally correct.

Visual and Audio Feedback

At minimum, your game should include:

- Distinct sprites for the player fish, NPC fish of varying sizes, and background/environment
- Visual scaling of entities as size changes (the player fish grows visibly)
- A directional flip so fish always face the direction they are swimming
- Sound effects for eating a fish, being eaten, and reaching a new size tier
- Background music for different game states

Assets

Free assets available at: OpenGameArt.org, itch.io, Kenney.nl. Ensure assets have appropriate licenses for educational use. Credit all assets in CREDITS.md.

Deliverables

1. The Game

An original playable Godot 4 project based on Feeding Frenzy, implementing all core mechanics and technical requirements. The game should be stable—no crashes, no game-breaking bugs, no orphaned nodes after game over. The core loop must work flawlessly.

2. GitHub Repository

Your complete Godot project must be hosted in a GitHub repository with clean commit history. Include:

- All Godot project files and scenes
- All assets with proper attribution in CREDITS.md
- README.md containing:
  - Game description and mechanical twist explanation
  - Controls documentation
  - Known issues or limitations
  - Asset credits and licenses
- Built game binary zipped in GitHub Releases section

Invite the instructor as a collaborator. Use semantic commit messages following the conventions from the syllabus.

3. Presentation (15 minutes)

Prepare a jam-style presentation covering:

- Live gameplay demonstration
- Architecture decisions and design patterns used
- Explanation of your mechanical twist
- Challenges faced and solutions
- Q&A from classmates and instructor

Class will vote on: Best Architecture, Best Twist, Best Polish. Winners receive +5% extra credit per category. This can stack up to +15%!

Weekly Progress Reports

Weekly progress reports during consultation hours are **strongly encouraged**. Groups that attend receive:

- Ongoing feedback on architecture
- Early detection of design issues
- Code review and refactoring guidance
- Up to +5% extra credit for consistent attendance

Groups that skip progress reports accept the risk of discovering major flaws during final presentations when fixes are costly or impossible, as well as forfeit the extra credit.

Grading Philosophy

This is not a competition to build the most spectacular fish game. The goal is to demonstrate that you can design and architect a system with emergent complexity—one that is flexible enough that adding your mechanical twist didn't require rewriting everything else.

A simple but well-architected game will score better than an ambitious but poorly architected one.

Focus on:

- Clean, maintainable code with proper design patterns
- Correct implementation of the size system and collision logic
- Meaningful mechanical twist, coherently integrated
- Professional commit history and documentation
- Excellent game "feel"—the fish should feel alive, not like sliders on a menu

Extra features (particle effects, elaborate menus, level transitions) are welcome after the foundation is solid, but will not compensate for poor architecture.

Pair Programming Expectations

- Both members must contribute to code and design
- Use Git branches, pull requests, code review
- Maintain clear communication and regular check-ins
- Document design notes in README.md
- Both members must speak during presentation

Individual accountability assessed through:

- GitHub commit history
- Presentation participation
- Consultation session involvement
- Peer evaluation (if issues arise)

See the syllabus for more details regarding the pair programming format.

Academic Honesty

The usage of Large Language Models (e.g., ChatGPT, Claude, Deepseek) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

You are expected to write your own code and understand every line you submit. During presentations and consultations, you must be able to explain and defend your architectural decisions and implementation choices.

Important Dates

Weekly Progress Reports

Progress reports are conducted during office hours on a first-come, first-served basis. Attendance is optional but **strongly encouraged**, and can earn up to +5% extra credit for the machine problem.

- **Week 1:** March 6, 2026
- **Week 2:** March 10, 2026
- **Week 3:** March 17, 2026
- **Week 4:** March 24, 2026

Submission of GitHub Repository

The GitHub repository for the machine problem must be submitted (with completed code and release build) on **April 1, 2026** (turn in an April Fools submission at your own risk). To submit, simply invite the instructor as a collaborator. Late submissions will be penalized with a -25% deduction. Commits dated after the deadline will not be considered during evaluation.

Final Presentation

Machine problem presentations will be conducted in a jam-style format during regular class hours. Both team members must be present and participate.

- **Presentation Date:** Tuesday, April 7, 2026
- **Duration:** 15 minutes per team (demo + Q&A)
- **Format:** Live gameplay demo, architecture showcase and explanation, voting

Note: Machine problems that are functional but exhibit significant architectural issues (e.g., spaghetti code, no FSM, hard-coded values, tight coupling) will receive a maximum grade of 3.00 (60/100) regardless of gameplay quality or presentation performance. Professional game development prioritizes maintainable, scalable systems over short-term functionality.

Grading Rubric

**Score may vary individually for component.*

Criteria	Excellent (90–100%)	Good (75–89%)	Fair (60–74%)	Poor (0–59%)
Architecture & Design (25%)	Clean separation of concerns, proper global and per-entity FSM implementation, signal-driven architecture, data-driven configuration, demonstrates sound design patterns	Minor architectural issues, most patterns implemented correctly, consistent structure	Significant architectural problems, inconsistent pattern usage, NPC behavior in wrong node, unclear structure	Spaghetti code, no clear architecture, monolithic scripts, violates OOP principles
Mechanical Twist (20%)	Creative, meaningful gameplay innovation that reshapes strategy; seamlessly integrated into core loop without breaking existing mechanics	Valid twist that changes gameplay meaningfully, decent integration	Minimal twist or poorly integrated, feels bolted on rather than designed	Cosmetic only, missing, or breaks core mechanics
Code Quality (10%)	Strongly typed signatures and members, excellent naming conventions, well-documented using Godot conventions, exported parameters for tunable values	Mostly typed, good naming, adequate documentation, some magic numbers present	Inconsistent typing/naming, sparse documentation, configuration buried in logic	Poor typing, unclear code, missing documentation, hard-coded values throughout
Individual Contributions* (20%)	Equitable commits with semantic messages, clear authorship, professional Git workflow (branches, PRs)	Mostly balanced contributions, adequate commit messages, basic Git usage	Uneven contributions visible in history, poor commit messages	One member dominates commits, nonsemantic messages, commit noise
Presentation* (25%)	Professional demo with smooth gameplay, clear architecture explanation, both members articulate design decisions, handles Q&A confidently and pitches game well	Good demo, adequate explanations of implementation, both members participate meaningfully, pitch is good	Rough demo with bugs, unclear explanations, uneven participation, struggles with questions, pitch is questionable	Unprepared, can't explain code or architecture, one member dominates, uncertain responses, pitch needs revision